

BANK MANAGEMENT SYSTEM

Project Report

A basic overview of a bank management system using Java can be summarized as follows: Java is used to create a user-friendly interface for banking operations, such as account management, fund transfers, balance inquiries, and transaction history. The system often integrates with a database management system like MySQL for secure data storage and retrieval. Java Swing is commonly used for the GUI, ensuring a smooth and efficient banking experience for both customers and bank staff.

Contents

1. Introduction - what this project is really about
 2. Why I built it (motivation and objectives)
 3. What is included and what is not (scope)
 4. Tools and technologies
 5. How the system is put together
 6. Features, explained one by one
 7. Screens and look of the application
 8. How data moves and how it is saved
 9. Rules the application enforces
 10. Compiling and running the project
 11. A full walkthrough with a sample user
 12. Problems faced and what I learned
 13. What could be improved later
 14. Closing thoughts
- Appendix - quick class reference
-

1. Introduction - what this project is really about

Almost everyone has used a bank app or an ATM. You check a balance before buying something, withdraw cash for the week, or glance at the last few transactions to see where money went. Those actions feel simple from the outside. Underneath, the system has to know who you are, protect the account with some kind of secret, update a balance carefully, and leave a trail so nothing just disappears.

This project, called the **Bank Management System**, is my attempt to rebuild that familiar experience as a small desktop application. It runs under a fictional brand name - **Prosperity National Bank** - and it is written entirely in Java using Swing for the windows and forms. There is no real bank on the other end of the wire. There is no internet requirement. What you get is a local program that still behaves like a mini bank counter or ATM menu: open an account, log in with a PIN, deposit, withdraw, use fast cash, check balance, read a mini statement, change the PIN, and log out.

I deliberately kept the setup light. Account details are stored in a file on disk using Java's object serialization. That means a reviewer or classmate can compile the code, run it, create an account, close the app, open it again, and still find the same balance and history. For a learning project, that continuity matters more than installing a full database server.

2. Why I built it (motivation and objectives)

A lot of college banking-system programs stop at a black console window with numbered menus. Those are useful for learning if-else and loops, but they do not feel like software people would actually click through. I wanted something closer to a product: screens that open and close, buttons with a consistent style, messages that say exactly what went wrong, and data that survives after you shut the window.

Building it also forced better habits than a single giant main method would. Money rules live on an Account object. Saving and loading live in one data-store class. Colors and button styles live in a shared theme helper.

Each dialog handles one job. That structure is the real learning outcome, even more than the feature list.

In concrete terms, the objectives were:

- **Complete GUI journey** - from "I am new here" to "I logged out safely," without falling back to the terminal.
 - **Solid OOP practice** - encapsulation on Account, a singleton store, separation between UI and business rules.
 - **Persistence without drama** - save accounts and history locally so demos do not reset every run.
 - **Believable banking constraints** - PIN checks, minimum opening deposit, no withdrawing more than you have.
 - **Code you can explain out loud** - one project file with clear sections, not a maze of unexplained classes.
-

3. What is included and what is not (scope)

3.1 Inside the boundary

The system is built around a single type of personal account. When someone signs up they give a full name, date of birth, ten-digit mobile number, a four-digit PIN (typed twice), and an opening deposit of at least 500. The system assigns an account number automatically, starting from 100100. After login, that person can deposit money, withdraw money (with PIN confirmation), use Fast Cash presets, view a balance summary, open a mini statement of the last ten transactions, and change the PIN. Logout returns to the login screen.

Exit closes the whole application after a confirmation.

3.2 Outside the boundary on purpose

This is a simulation for learning, not core banking software. I did not add PIN hashing or encryption, network multi-user access, a staff/admin console, loans, interest calculation, or transfers between two customers. Those are honest next steps, but adding them now would have stretched the project past "clear demo" into "half-finished product." I preferred a finished small world over an unfinished big one.

4. Tools and technologies

Every piece below ships with a normal JDK. That was intentional - lab machines and personal laptops should run the project without hunting for extra installers.

Technology	How it is used in this project
Java (JDK 8 or newer)	Main language; collections for the account map and transaction list; basic I/O
javax.swing	Frames, dialogs, buttons, text fields, password fields, option panes, scroll panes
java.awt	Layouts (Border, Grid, GridBag), colors, fonts, cursors, insets and padding
Object serialization	Write and read the full set of accounts plus the next account number to bank_data.ser
java.time	Stamp each transaction with a readable date and time (dd-MM-yyyy HH:mm:ss)

5. How the system is put together

If you open the source top to bottom, you will see sections rather than a random pile of classes. Here is how I think about the design when I explain it.

5.1 The account (the heart of the money logic)

The **Account** class is a serializable model. It keeps the account number, name, date of birth, mobile, PIN, balance, and a list of transaction strings. When an account is created, it immediately logs an ACCOUNT OPENED line with the opening balance. Deposit adds to the balance and logs. Withdraw checks funds first; if there is not enough money it returns false and changes nothing. Change PIN verifies the old PIN before replacing it. In other

words, the object protects its own invariants. Button code does not secretly edit the balance field from outside.

5.2 The data store (memory + disk)

BankDataStore is a singleton - one shared instance for the whole app. On first access it tries to load `bank_data.ser`. If the file is missing or unreadable, it starts with an empty map and a note on the console, instead of crashing. Creating an account takes the next number, puts the account in a `HashMap`, and saves. Login looks accounts up by number. Every successful deposit, withdrawal, or PIN change from the UI ends with an explicit save, so a normal exit does not wipe the afternoon's work.

5.3 The screens (what the user actually sees)

Presentation is split into frames and dialogs. **LoginFrame** and **SignupFrame** handle getting into the system. **DashboardFrame** is the home base after a successful login - it shows the welcome name, account number, live balance, and the main action buttons. Operations open as modal dialogs: Deposit, Withdraw, Fast Cash, Mini Statement, Change PIN. Modal means the user finishes or cancels that task before poking other buttons, which matches how ATMs usually feel.

5.4 Shared styling (so it looks like one product)

UITheme is a small utility class with the color palette, fonts, and factory methods for primary, accent, and danger buttons, plus text fields and standard info/error/confirm popups. Without this, every screen starts inventing its own blue and its own padding. With it, the app reads as one design even though many classes draw windows.

The entry class **BankManagementSystem** only does two quiet jobs: try to apply the system look-and-feel so window chrome matches the OS, and open the login screen on Swing's Event Dispatch Thread. That is the recommended way to start Swing UIs and avoids odd painting glitches later.

6. Features, explained one by one

Below is the feature set in the order a new user is likely to meet it, with a bit more detail than a bullet list usually gets.

Open a new account. Signup collects identity-style fields and security fields together. The mobile number must be exactly ten digits. The PIN must be exactly four digits and must match the confirmation box. The opening deposit must parse as a number and must be at least 500 - I did not want accounts that start at zero and feel fake. On success, a dialog shows the new account number. That number is the user's login ID forever in this demo, so the message tells them to note it down.

Login. The user types account number and PIN. Empty fields, non-numeric account numbers, unknown accounts, and wrong PINs each get their own clear error. A correct pair opens the dashboard and closes the login window. Pressing Enter in the PIN field also submits, which is a small thing people expect from any login form.

Deposit. A simple dialog asks for an amount. Anything that is not a positive number is rejected. A valid deposit updates the balance, writes a DEPOSIT line into the history, saves the file, refreshes the gold balance label on the dashboard, and shows the new total.

Withdraw. Same idea as deposit, but stricter. The user must re-enter the PIN even though they are already logged in - a second gate before cash leaves. If the PIN is wrong, nothing moves. If the amount is larger than the balance, the account's withdraw method refuses the operation and the UI says "Insufficient balance."

Fast Cash. This is the ATM-style shortcut. Instead of typing, the user picks 500, 1000, 2000, 5000, 10000, or 20000. Under the hood it still calls the same withdraw logic, so the "do I have enough money?" rule stays in one place. That reuse mattered to me: two UIs, one rule.

Balance enquiry. A quick popup lists holder name, account number, and current balance. No extra typing.

Useful when you only want confirmation, not a full history.

Mini statement. Shows up to the last ten transaction lines, newest first, in a monospaced text area so amounts line up. Each line carries a timestamp, a type such as DEPOSIT or WITHDRAW, the amount (or a dash for PIN changes), and the running balance after that event.

Change PIN. Needs the current PIN, a new four-digit PIN, and a matching confirmation. On success the change is logged (without printing the PIN itself) and saved to disk.

Logout and Exit. Both ask "are you sure?" Logout reopens the login screen. Exit ends the program.

Confirmations sound minor until the first time they stop an accidental click during a demo.

Feature	Short summary
Open account	KYC-style fields + PIN + minimum 500 deposit; auto account number
Login	Account number + PIN with specific error messages
Deposit / Withdraw	Validated amounts; withdraw also re-checks PIN and balance
Fast Cash	One-tap preset withdrawals using the same withdraw path
Balance / Statement	Instant summary popup, or last 10 timestamped ledger lines
Change PIN / Logout	Safe PIN update; confirmed logout or full exit

7. Screens and look of the application

Banking software does not need loud design. It needs to feel steady. I picked a deep navy for headers and primary buttons, a warm gold for the live balance and secondary calls-to-action, a soft blue-gray for backgrounds, and a muted red only for Exit so it never competes with everyday actions.

Login is a calm centered form under a bank-style header bar. Signup is a longer form in a scroll pane because there are more fields. The dashboard is where the design does the most work: a dark strip shows the account number and the balance in large gold text, and below it an eight-button grid gives equal weight to the main operations. Dialogs stay

compact and focused on one question - "how much?" or "what is your PIN?" - then get out of the way.

Fonts lean on Segoe UI where the system has it. The mini statement uses a monospaced font so columns of numbers are easy to scan. Buttons share bold labels, comfortable padding, hand cursors, and no ugly focus rings. None of that changes the business logic, but it changes whether the project feels finished when someone clicks through it for the first time.

8. How data moves and how it is saved

It helps to follow one happy path in slow motion. Take a deposit:

1. The user clicks **Deposit** on the dashboard. A modal Deposit dialog opens.
2. They type an amount and press the Deposit button inside the dialog.
3. The dialog parses the text. If it is not a valid positive number, an error appears and nothing else happens.
4. On success, `account.deposit(amount)` increases the balance and appends a timestamped DEPOSIT line to the account's transaction list.
5. `BankDataStore.getInstance().save()` writes the whole accounts map and the next-account counter into `bank_data.ser`.
6. A callback refreshes the dashboard's balance label so the user sees the new figure without reopening the window.
7. A short success message shows the new balance, and the dialog closes.

Withdraw is the same pipeline with two extra gates: verify PIN, then verify sufficient balance. Login never writes to disk on failure - failed attempts are read-only checks, which is what you want. I chose serialization over JDBC because the audience for this project is often a lab PC or a quick GitHub clone. Asking everyone to install and wire MySQL would add friction without teaching the GUI and OOP lessons any better. The trade-off is obvious and acceptable for a demo: great for single-user local use, not a stand-in for a real multi-user database.

9. Rules the application enforces

Trust in a money-related app is mostly about what it refuses to do. These are the hard stops built into the forms and the Account methods:

Rule	What the user experiences
All signup fields required	Empty form cannot create an account
Mobile = 10 digits	Letters or wrong length are rejected with a plain message
PIN = 4 digits	Enforced on create and on change

PIN confirmation	Create and change both demand a matching second entry
Minimum opening deposit 500	Blocks empty or token accounts
Amounts greater than 0	Zero or negative deposit/withdraw is refused
No overdraft	withdraw() returns false; UI shows insufficient balance
PIN on custom withdraw	Even a logged-in user must confirm before cash out
Numeric account number at login	Non-numbers never hit the account map

10. Compiling and running the project

You need a JDK (8 or newer) and a computer with a graphical desktop. From the folder that contains the source file:

```
javac
BankManagementSystem.java
java BankManagementSystem
```

The first run will not find a data file yet - that is normal. After the first successful account creation, `bank_data.ser` appears in the working directory. Keep that file if you want your demo data to survive. Delete it when you want the bank wiped clean; account numbers will start again at 100100. Because this is a GUI program, it will not do much on a headless server without a display.

11. A full walkthrough with a sample user

Abstract feature lists are easier to believe when you follow one person through the app. Meet Aisha.

Aisha starts the program and lands on the Prosperity National Bank login screen. She has never banked here, so she chooses **Create an Account**. She enters her name, date of birth, mobile number, an opening deposit of 2000, and the PIN 4821 twice. The app accepts the form and shows a message she cannot ignore: her account number is **100100**. She writes it down.

Back at login she signs in with 100100 and 4821. The dashboard greets her by name. The balance strip shows 2000.00 in gold. She deposits another 500; the label ticks to 2500.00 and a success dialog confirms it. Later she tries Fast Cash for 1000. It works, and the balance becomes 1500.00. Curious about the paper trail, she opens Mini Statement and sees three lines - account opened, deposit, withdraw - each with a time and a running balance. Before she leaves for the day she changes her PIN, logs out through the

confirmation prompt, and the login screen returns, ready for the next session. Nothing in that path required a database administrator or a command-line menu.

12. Problems faced and what I learned

A few snags showed up while building this, and each one left a lesson I would reuse.

Mixing UI and money math. Early on, it was tempting to add and subtract balances directly inside button listeners. That got messy fast. Moving deposit and withdraw onto Account cleaned the dialogs and made the rules obvious: if you want to change money, you ask the account.

Updating the dashboard after a dialog. Dialogs are separate windows. They needed a polite way to say "the balance changed." Passing a `Runnable onSuccess` callback from the dashboard into each money dialog was enough. No event-bus framework required.

Broken or missing save files. A half-written or outdated `.ser` file can fail on load. Catching that and starting fresh with a short console message is kinder in a lab than a full stack trace.

Vague errors frustrate people. "Invalid input" helps no one. "Mobile number must be exactly 10 digits" or "Minimum opening deposit is 500.00" turns a failed demo into a fixable mistake.

Style drift is real. The third dialog is when copy-pasted colors start to disagree. `UITheme` paid for itself the moment every button could be created in one line with the same padding and palette.

13. What could be improved later

If this project grew beyond a teaching demo, I would not start by adding random features. I would strengthen the foundations first, then widen the product:

- Store PIN hashes (for example with a slow hash) instead of plain text inside the serialized file.
- Move persistence to SQLite or MySQL when multi-user or heavier reporting is needed.
- Add fund transfer between two account numbers with a clear double-sided log.
- Build a small admin view to search customers or review activity.
- Export statements to PDF or CSV so records can leave the app.
- Add automated tests around Account and BankDataStore so refactors stay safe.

None of those are admissions that the current project is incomplete for its goal. They are the natural staircase after a vertical slice already works end to end.

14. Closing thoughts

The Bank Management System set out to be a friendly, complete desktop demo of everyday banking actions and that is what it is. From the first login screen to the last mini-statement

line, it shows how Java Swing, careful object boundaries, and simple file persistence can combine into something that feels intentional rather than academic for its own sake.

I am more attached to the habits than to any single button: validate early, let the model guard its own data, tell the user exactly what failed, save when a real change happens, and keep the visual language consistent.

Those habits travel with you whether the next build is another Swing tool, a web API, or a mobile client.

It remains a simulation. It will not move real money, and it should not. But as a learning artifact and a portfolio piece, it tells a clean story: open an account, trust the PIN gate, move funds carefully, read the ledger, and lock the door on the way out.

THANK YOU
